



Today's agenda

↳ next greater element +4

↳ largest rectangle area in a histogram

↳ largest area rectangle in a binary → H.W

← LIFO

← name

→ `Stack<Integer> st = new Stack<>();`

AlgoPrep



→ next greater element

Q) NGE on right

↳ Given arr[], for every index find the greater element on right side.

	0	1	2	3	4	5
Ex: arr[]:	4	9	5	8	2	10
ans[]:	9	10	8	10	10	-1

// idea 1

↳ nested for loop.

	0	1	2	3	4	5
arr[]:	4	9	5	8	2	10
ans[]:	9	10	8	10	10	-1

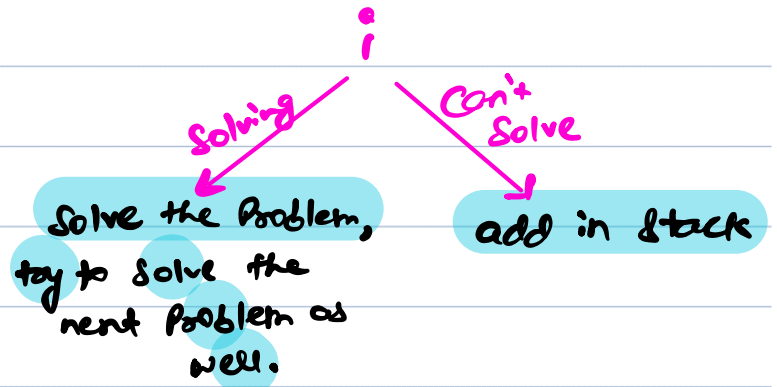
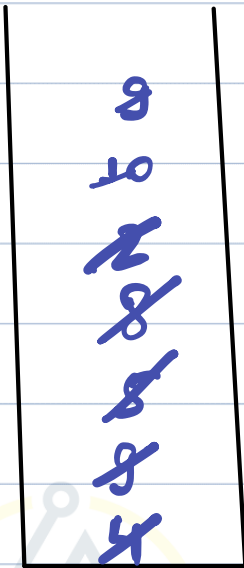
T.C: $O(N^2)$

S.C: $O(1)$



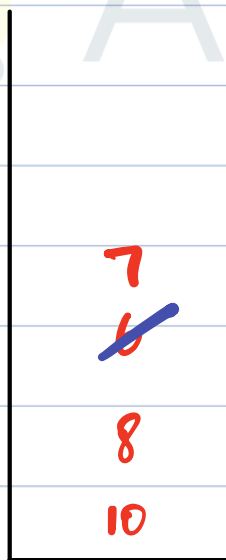
Idea 2

arr[7]: 4 9 5 8 2 10 9
ans[7]: 9 10 8 10 10 -1 -1



Obs 1:

dec.
order

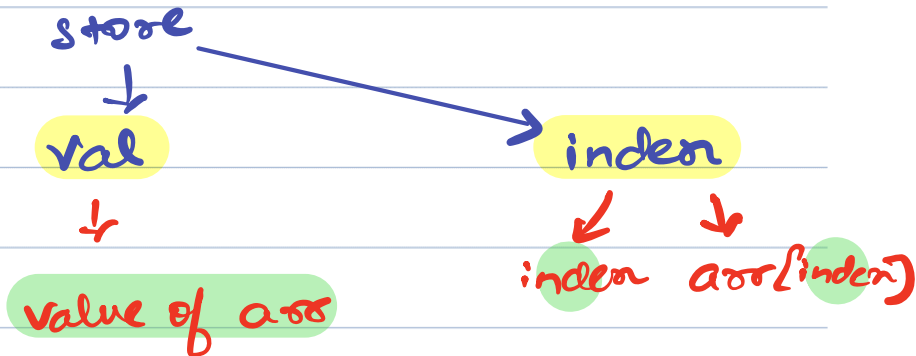
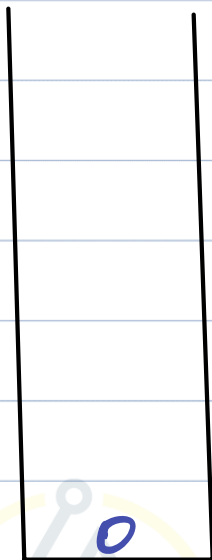


10 8 6 7 5
7



obs 2%

		0	1	2	3	4	5	6
arr[i]:		4	9	5	8	2	10	9
ans[i]:							-1	-1



// Pseudo code

T.C: $O(N)$

S.C: $O(N)$

```

int[] nextRight (int arr[]) {
    int[] ans;
    Stack<Integer> st;
    for (int i=0; i<N; i++) {
        while (st.size() > 0 && arr[i] > arr[st.top()]) {
            int idn = st.pop();
            ans[idn] = arr[i];
        }
        st.push(i);
    }
    while (st.size() > 0) {
        int idn = st.pop();
        ans[idn] = -1;
    }
}
  
```



```
int[] nextRight (int arr[]) {  
    int[] ans;
```

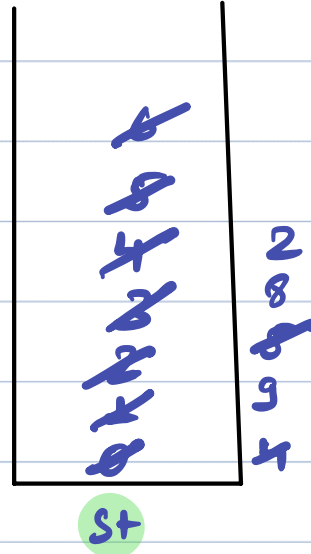
```
    for (int i=0; i<N; i++) {  
        while (st.size() > 0 && arr[i] > arr[st.top()]) {  
            int idn = st.pop();  
            ans[idn] = arr[i];  
        }  
        st.push(i);  
    }
```

```
    while (st.size() > 0) {  
        int idn = st.pop();  
        ans[idn] = -1;  
    }
```

i

	0	1	2	3	4	5	6
arr[i]:	4	9	5	8	2	10	9

ans[i]:	9	10	8	10	10	-1	-1
---------	---	----	---	----	----	----	----



$i=0 \rightarrow 0$
 $i=1 \rightarrow 0/1$
 $i=2 \rightarrow 0/1/2$
 $i=3 \rightarrow 0/1/2$
 $i=4 \rightarrow 0/1/2/3$
 $i=5 \rightarrow 0/1/2/3/4$
 $i=6 \rightarrow 0/1/2/3/4/5$

$i=N \rightarrow \frac{0}{N}$
 $O(N)$



Q) NSE on right
 → next smaller element

↳ Given arr[], for every index find the smaller element on right side.

arr[]: 4 9 5 8 2 10 9
 ans[]:

// Pseudo code

```
int[] nseonright (int arr[]) {
    int[] ans;
    Stack<Integer> st;
    for (int i=0; i<arr.length; i++) {
        while (st.size() > 0 && arr[i] < arr[st.top()]) {
            int idn = st.pop();
            ans[idn] = arr[i];
        }
        st.push(i);
    }
    while (st.size() > 0) {
        int idn = st.pop();
        ans[idn] = -1;
    }
}
```

T.C: $O(N)$

S.C: $O(N)$



Q) NGE on left

↳ Given arr[], for every index find the greater element on left side.

4
9
5
8
2
10
9

↳ Run the loop backwards.

arr[]: 4 9 5 8 2 10 9
-1 -1 9 9 8 -1 10

// Pseudo code

```
int[] ngeonleft (int arr[]) {
```

```
    int[] ans;
```

```
    Stack <Integer> st;
```

```
    for (int i = arr.length - 1; i >= 0; i--) {
```

```
        while (st.size() > 0 && arr[i] > arr[st.top()])
```

```
            int idn = st.pop();
```

```
            ans[idn] = arr[i];
```

```
        }
```

```
        st.push(i);
```

```
    }
```

```
    while (st.size() > 0) {
```

```
        int idn = st.pop();
```

```
        ans[idn] = -1;
```

```
    }
```

```
}
```

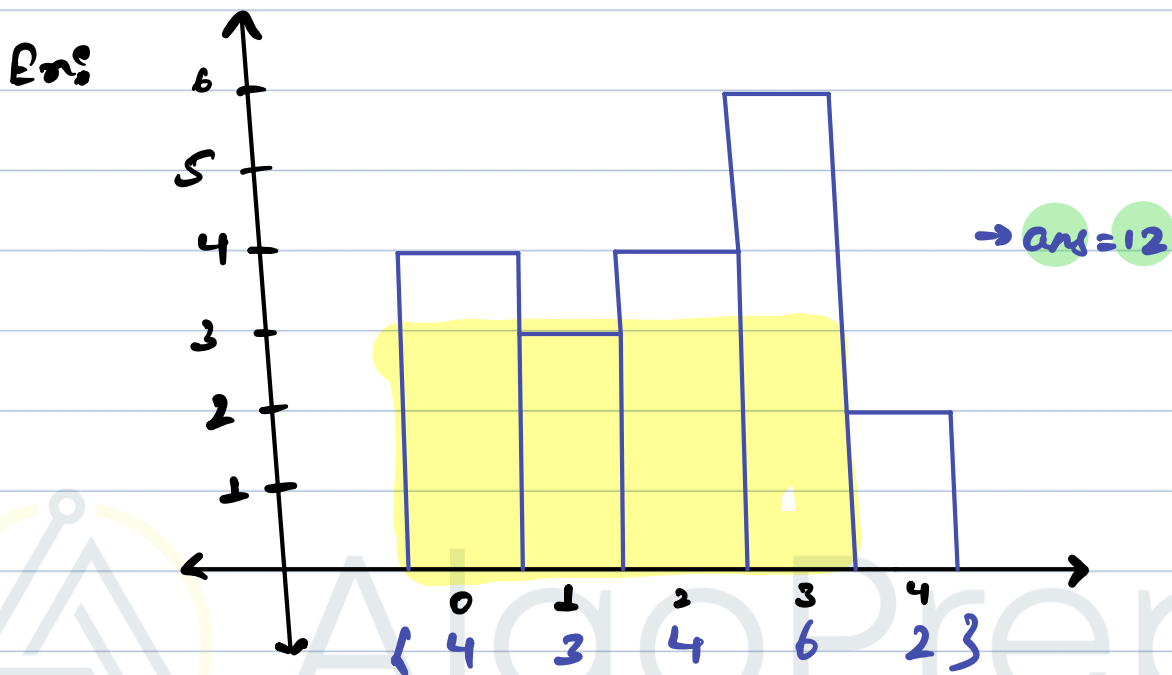
T.C: $O(n)$

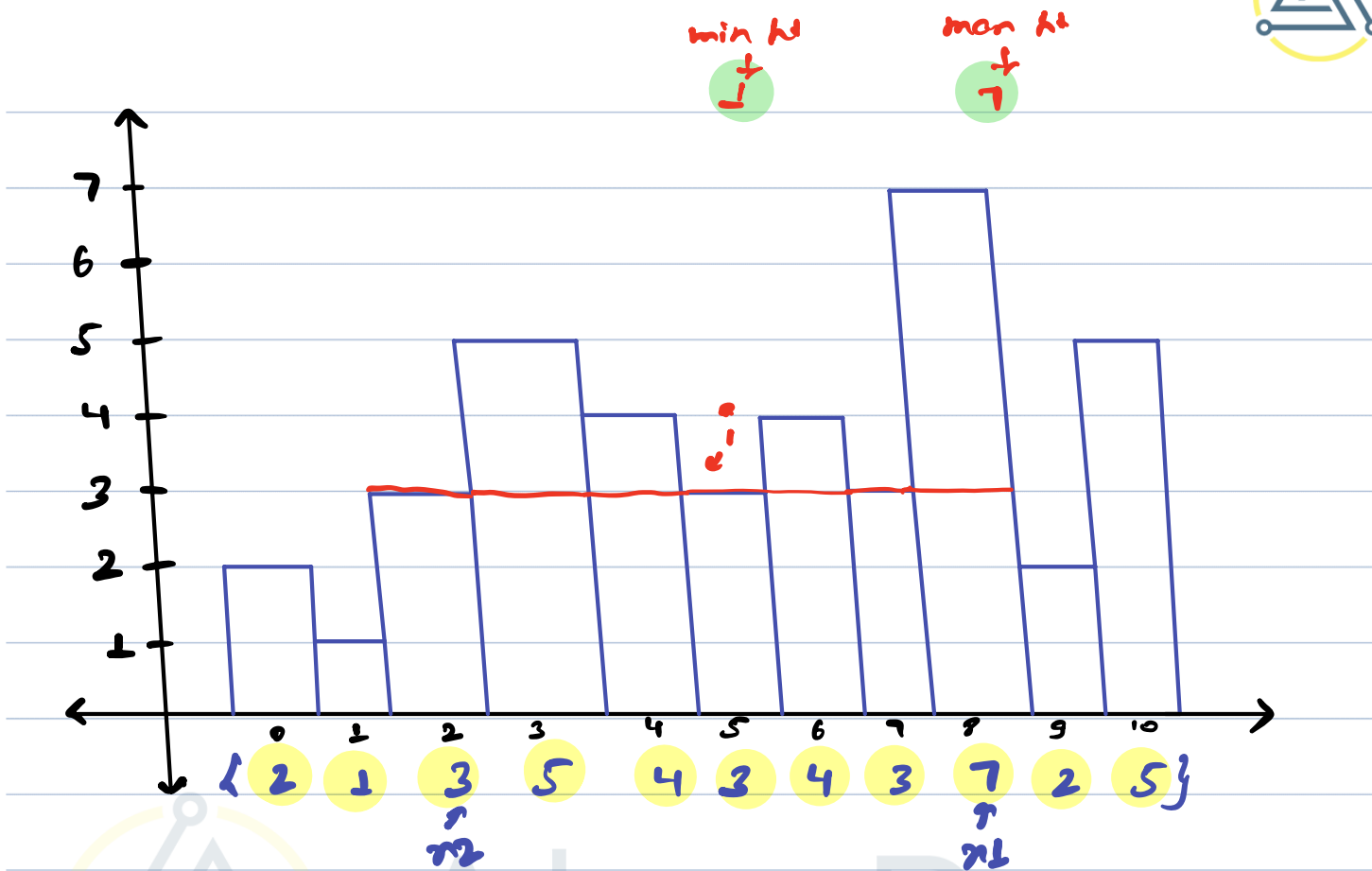
S.C: $O(n)$



Q) Largest area rectangle in the given histogram.

↳ Given $arr[n]$ representing histogram, what is the largest area rectangle you can create.



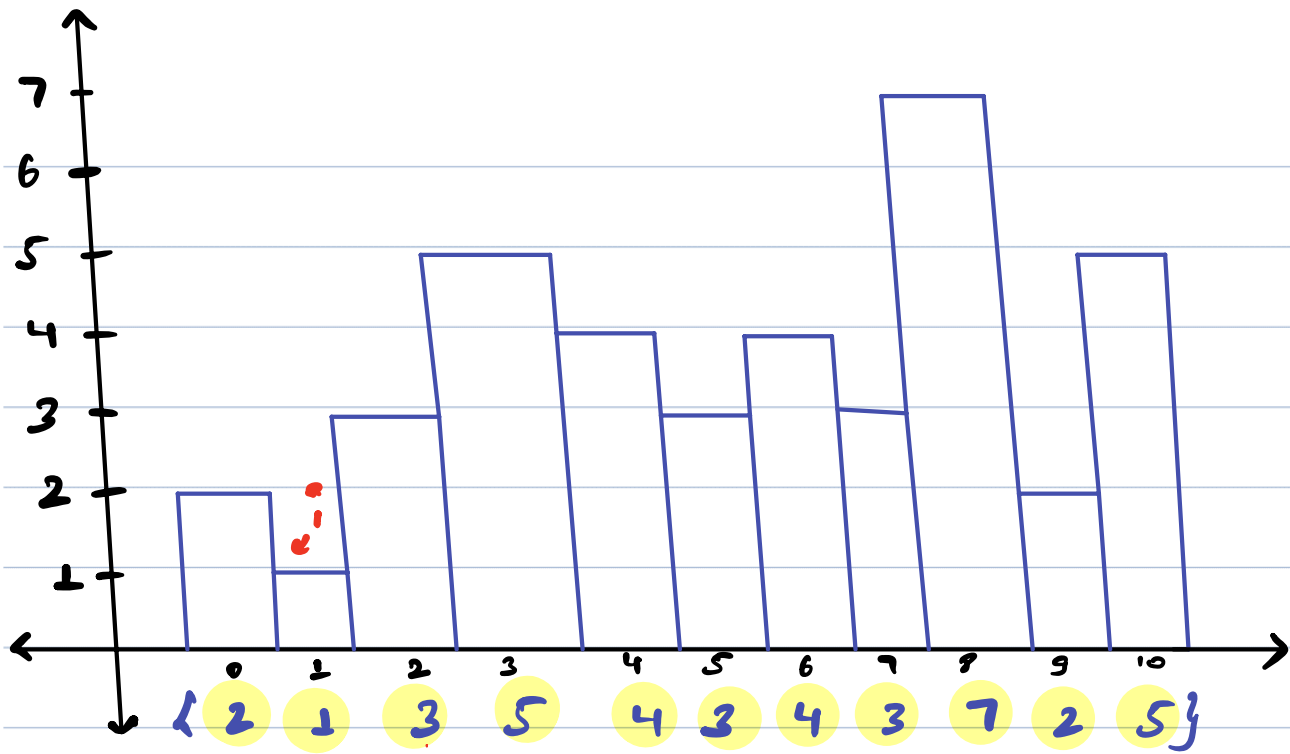


Obs! check for heights that are present in histogram.

right idn till which you can expand: (next smaller idn on right - 1)
↳ n1

left idn till which you can expand: (next smaller idn on left + 1)
↳ n2

$$\text{width} = (n1 - n2 + 1)$$



NSE idn on:
right

1 11 9 4 5 9 7 9 9 11 11

NSE idn on:
left

1 1 1 2 2 1 5 1 7 1 9

height

width

area

i:2

3

n1=8
n2=2 ≈ 7

21

i:3

5

n1=3
n2=3 ≈ 1

5

i:4

4

n1=4
n2=3 ≈ 2

8

i:1

1

n1=10
n2=0



//PSuedo code

```
int LAH (int[] height) {
```

```
    int NSEonright[] = func1(height);
```

```
    int NSEonleft[] = func2(height);
```

```
    int ans = -∞;
```

T.C: $O(N)$

S.C: $O(N)$

```
    for (int i=0; i<N; i++) {
```

```
        int ht = height[i];
```

```
        int n1 = NSEonright[i] - 1;
```

```
        int n2 = NSEonleft[i] + 1;
```

```
        int wd = n1 - n2 + 1;
```

```
        int area = ht * wd;
```

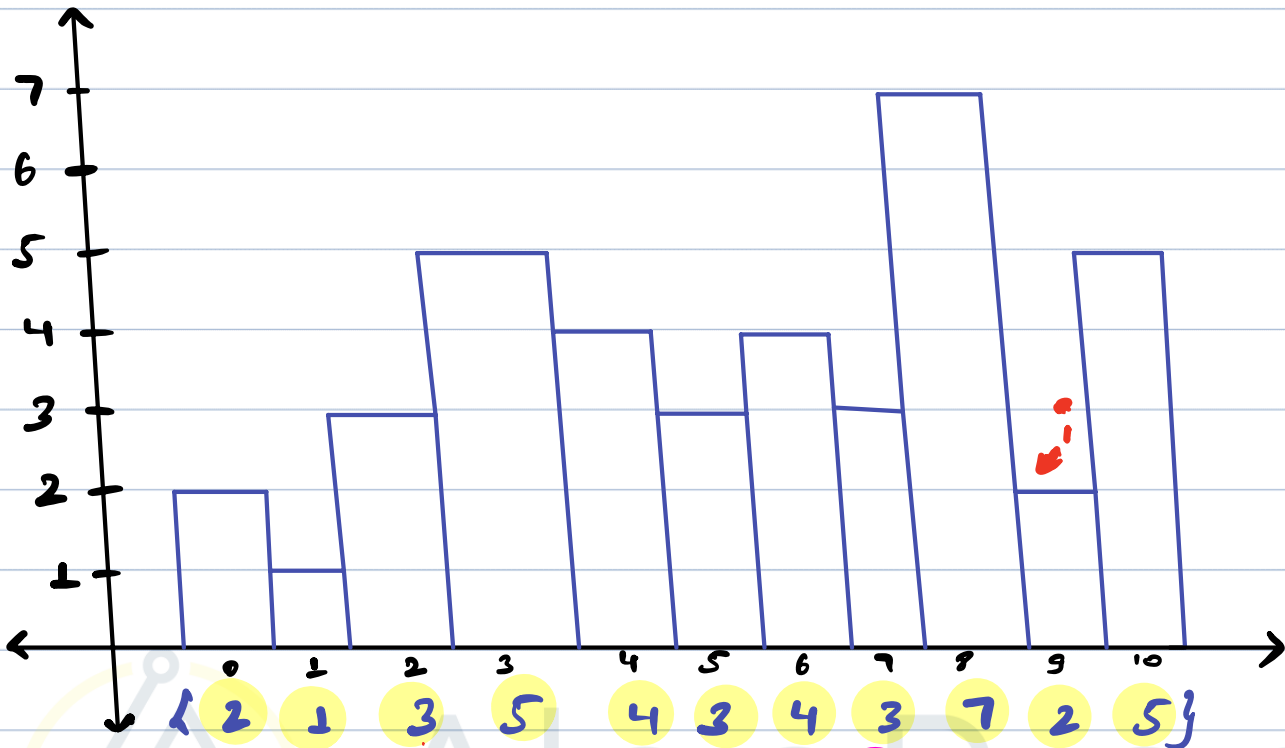
```
        ans = Math.max(ans, area);
```

```
    }
```

```
    return ans;
```

```
}
```

Single iteration



↓

4

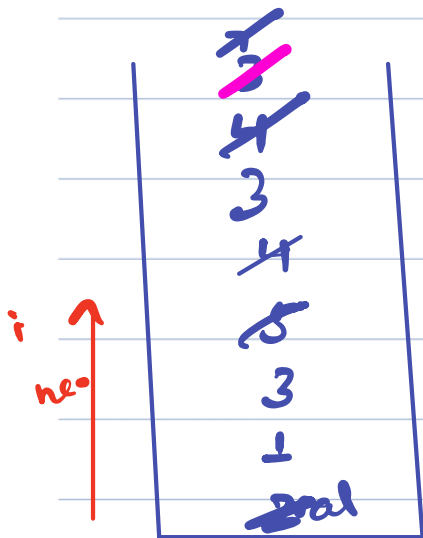
3

3

2

2

→ solve for nse on right.



Obs: while solving nse on right, nse on left is just below.



1/ Pseudo code

```
int LAH (int arr[N]) {
```

```
    Stack<Integer> S = new Stack<>();
```

```
    int ans = -∞;
```

```
    for (int i=0; i<arr.length; i++) {
```

```
        while (S.size() > 0 && arr[i] < arr[S.peek()]) {
```

```
            int tbs = S.pop();
```

```
            int nst = i;
```

```
            int nsl = nst - 1;
```

```
            int nsl = -1;
```

```
            if (S.size() == 0) { nsl = -1; }
```

```
            else { nsl = S.pop(); }
```

```
            int n2 = nsl + 1;
```

```
            int area = height[tbs] * (n2 - n1 + 1)
```

```
            ans = max(ans, area);
```

```
        }
```

```
    }
```

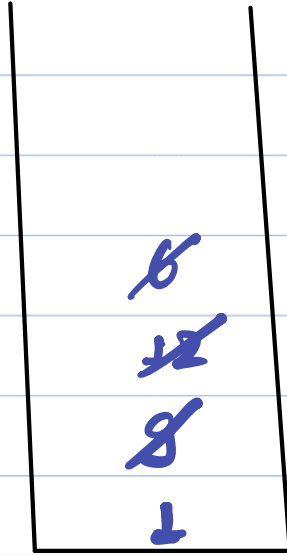
```
    return ans;
```

T.C: $O(N)$
S.C: $O(N)$



0 1 2 3 4
→ { 1 9 12 6 3 }
6 6 3

→ rise on right



AlgoPrep